

0.10.2. Tout l'alphabet, avec poserS — doublé, deux positions

Tout l'alphabet deux fois : lignes 0 et 1, puis lignes 3 et 4.

1. Ce qu'il faut comprendre

C'est le 0.10 et le 0.10.1 réunis : l'alphabet du 0.10 (lignes 0 et 1), puis celui du 0.10.1 (lignes 3 et 4).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poserS>` → [0.9.3. La fonction poserS\(\) — poser, et passer à la ligne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1013 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	139 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
Diviser	17 o	← poser() — divise — le processeur ne sait pas le faire
Reste	10 o	← poser() — le reste d'une division
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (poserS() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 2	96 o sur place, dans main() + AdresseCase (10 o), Diviser (17 o), Reste (10 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
poserS()	× 2	0 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <poserS>    // pose une tuile, et passe à la ligne au bord
```

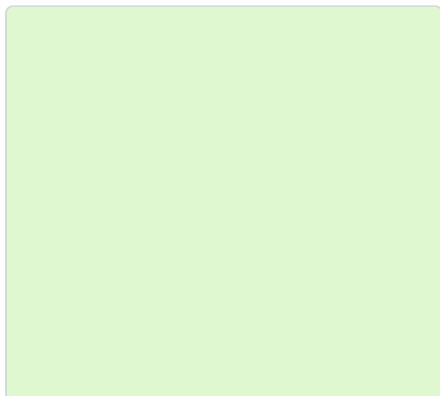
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // 1. Lignes 0 et 1 (le 0.10)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poserS(i, 0, ALPHABET[i]); // poserS passe tout seul à la ligne 1
    }
    // 2. Lignes 3 et 4 (le 0.10.1)
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {
        poserS(i, 3, ALPHABET[i]); // poserS passe tout seul à la ligne 4
    }

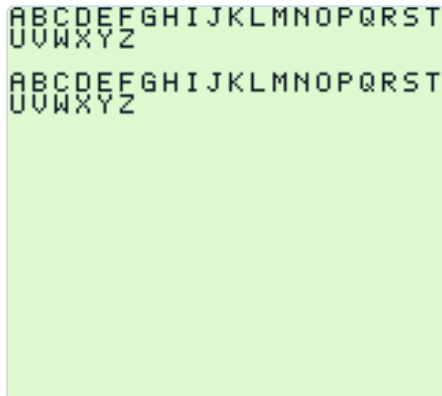
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Tout l'alphabet deux fois : en haut, et trois lignes plus bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ lignes 0 et 1

✓ lignes 3 et 4

Ensuite : **0.10.3. La fonction `textS()` — un texte qui passe à la ligne** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++